

Ejercicios de Árboles de decisión

Jan Žurek y Guido D’Inverno

05/2026

Contents

Ejercicio 1 :Árbol de decisión de dataset ‘Carseats’	1
1.1. Análisis previos	1
1.2 Modelo de entrenamiento pt1	3
1.3 Modelo de entrenamiento pt2	6
1.4 Cálculo de las predicciones sobre el conjunto de datos	11
Ejercicio 2	12
2.1. Carga y exploración inicial de los datos	12
2.2. Análisis Exploratorio de Datos (EDA)	14
2.4. Construcción del modelo	18
2.5. Visualización del árbol	18
2.6. Evaluación del modelo (Predicción)	20

Ejercicio 1 :Árbol de decisión de dataset ‘Carseats’

1.1. Análisis previos

```
library(ISLR)

Carseats <- Carseats
pander(head(Carseats))
```

Table 1: Table continues below

Sales	CompPrice	Income	Advertising	Population	Price	ShelveLoc
9.5	138	73	11	276	120	Bad
11.22	111	48	16	260	83	Good
10.06	113	35	10	269	80	Medium
7.4	117	100	4	466	97	Medium
4.15	141	64	3	340	128	Bad
10.81	124	113	13	501	72	Bad

Age	Education	Urban	US
42	17	Yes	Yes
65	10	Yes	Yes
59	12	Yes	Yes
55	14	Yes	Yes

Age	Education	Urban	US
38	13	Yes	No
78	16	No	Yes

Datos cargados correctamente.

```
# Verificación de valores faltantes
pander(colSums(is.na(Carseats)))
```

Table 3: Table continues below

Sales	CompPrice	Income	Advertising	Population	Price	ShelveLoc
0	0	0	0	0	0	0

Age	Education	Urban	US
0	0	0	0

No tenemos valores NA.

```
Carseats$Sales[Carseats$Sales>8]<-"Altas"
Carseats$Sales[Carseats$Sales<=8]<-"Bajas"
Carseats$Sales<-as.factor(Carseats$Sales)
str(Carseats)

## 'data.frame': 400 obs. of 11 variables:
## $ Sales : Factor w/ 2 levels "Altas","Bajas": 1 1 1 2 2 1 2 1 2 2 ...
## $ CompPrice : num 138 111 113 117 141 124 115 136 132 132 ...
## $ Income : num 73 48 35 100 64 113 105 81 110 113 ...
## $ Advertising: num 11 16 10 4 3 13 0 15 0 0 ...
## $ Population : num 276 260 269 466 340 501 45 425 108 131 ...
## $ Price : num 120 83 80 97 128 72 108 120 124 124 ...
## $ ShelveLoc : Factor w/ 3 levels "Bad","Good","Medium": 1 2 3 3 1 1 3 2 3 3 ...
## $ Age : num 42 65 59 55 38 78 71 67 76 76 ...
## $ Education : num 17 10 12 14 13 16 15 10 10 17 ...
## $ Urban : Factor w/ 2 levels "No","Yes": 2 2 2 2 2 1 2 2 1 1 ...
## $ US : Factor w/ 2 levels "No","Yes": 2 2 2 2 1 2 1 2 1 2 ...
```

Estructura de datos preparada correctamente para generar un modelo de clasificación.

```
summary(Carseats)
```

```
## Sales CompPrice Income Advertising Population
## Altas:164 Min. : 77 Min. : 21.00 Min. : 0.000 Min. : 10.0
## Bajas:236 1st Qu.:115 1st Qu.: 42.75 1st Qu.: 0.000 1st Qu.:139.0
## Median :125 Median : 69.00 Median : 5.000 Median :272.0
## Mean :125 Mean : 68.66 Mean : 6.635 Mean :264.8
## 3rd Qu.:135 3rd Qu.: 91.00 3rd Qu.:12.000 3rd Qu.:398.5
## Max. :175 Max. :120.00 Max. :29.000 Max. :509.0
## Price ShelveLoc Age Education Urban
## Min. : 24.0 Bad : 96 Min. :25.00 Min. :10.0 No :118
## 1st Qu.:100.0 Good : 85 1st Qu.:39.75 1st Qu.:12.0 Yes:282
## Median :117.0 Medium:219 Median :54.50 Median :14.0
```

```
## Mean      :115.8          Mean      :53.32    Mean      :13.9
## 3rd Qu.   :131.0          3rd Qu.   :66.00    3rd Qu.   :16.0
## Max.      :191.0          Max.      :80.00    Max.      :18.0
## US
## No :142
## Yes:258
##
##
##
##
```

De un breve análisis descriptivo se desprende que al menos el 25 % de las tiendas no invierte nada en advertising .

En la categoría objetivo (Sales), se observa que hay muchas más bajas que altas.

1.2 Modelo de entrenamiento pt1

```
# CONFIGURACIÓN DE K-FOLD CROSS VALIDATION (10 Folds)
set.seed(12345)
entrenamiento <- trainControl(
  method = "cv",
  number = 10,
  classProbs = TRUE,
  summaryFunction = twoClassSummary, # Permite calcular AUC, Sensibilidad y Especificidad
  savePredictions = "final",
  verboseIter = TRUE
)
```

```
print(modelo)
```

```
## CART
##
## 400 samples
## 10 predictor
## 2 classes: 'Altas', 'Bajas'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 359, 360, 360, 361, 359, 360, ...
## Resampling results across tuning parameters:
##
##   cp          ROC          Sens          Spec
## 0.04573171 0.7247696 0.6172794 0.8184783
## 0.10975610 0.6808654 0.4628676 0.8815217
## 0.28658537 0.5454444 0.1463235 0.9445652
##
## ROC was used to select the optimal model using the largest value.
## The final value used for the model was cp = 0.04573171.
```

El algoritmo ha probado tres versiones diferentes del árbol; el ganador es un modelo con $cp = 0.0457$.

Esto se debe a que tiene una curva ROC más alta que todas las demás; tenemos una buena capacidad discriminatoria, aunque podría mejorarse.

La sensibilidad es buena: el modelo identifica correctamente una tienda ‘Altas’ el 62 % de las veces.

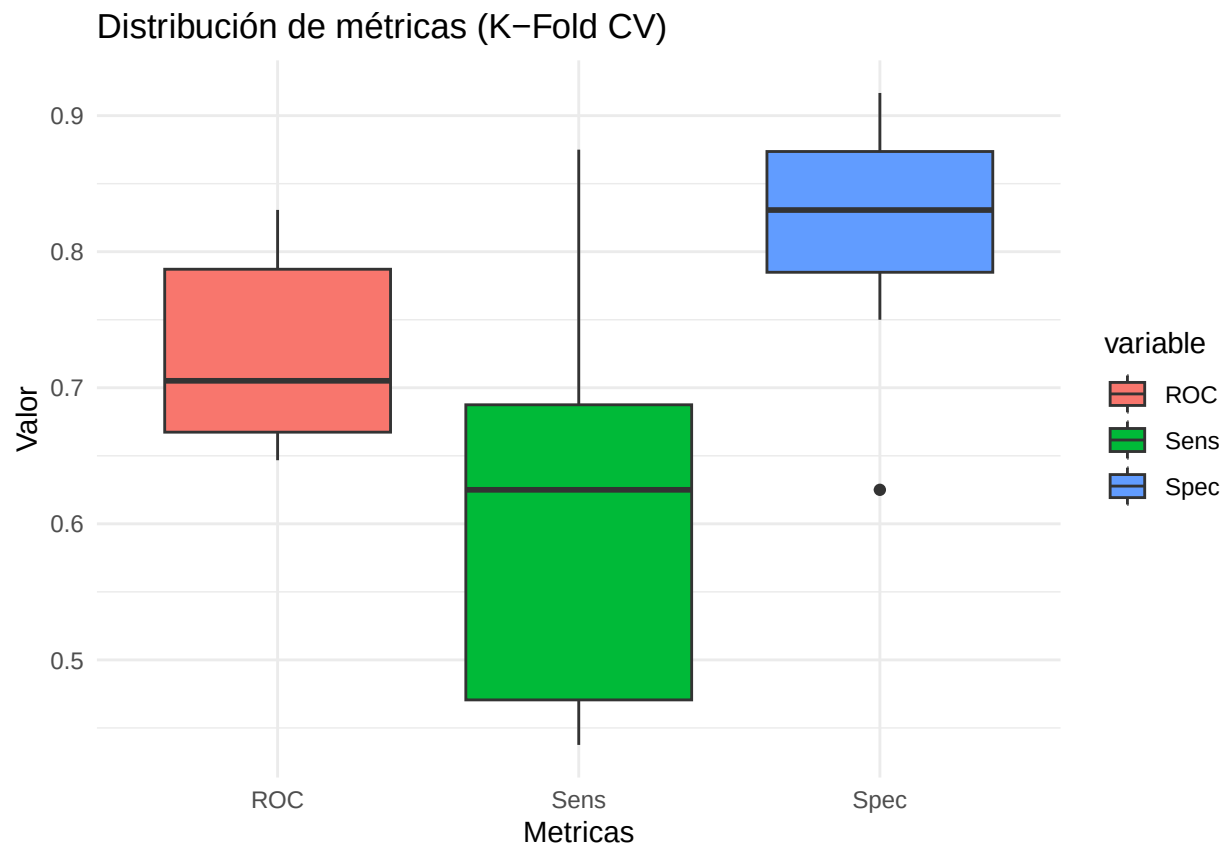
La especificidad es excelente; prácticamente es muy bueno a la hora de determinar cuándo una tienda es 'Bajas' (el 82 % de las veces).

Diagrama de caja de las métricas de rendimiento (ROC, sensibilidad, especificidad) obtenidas en los 1

```
df_plot <- melt(modelo$resample[, c("ROC", "Sens", "Spec")])
```

#Grafo Box-plot

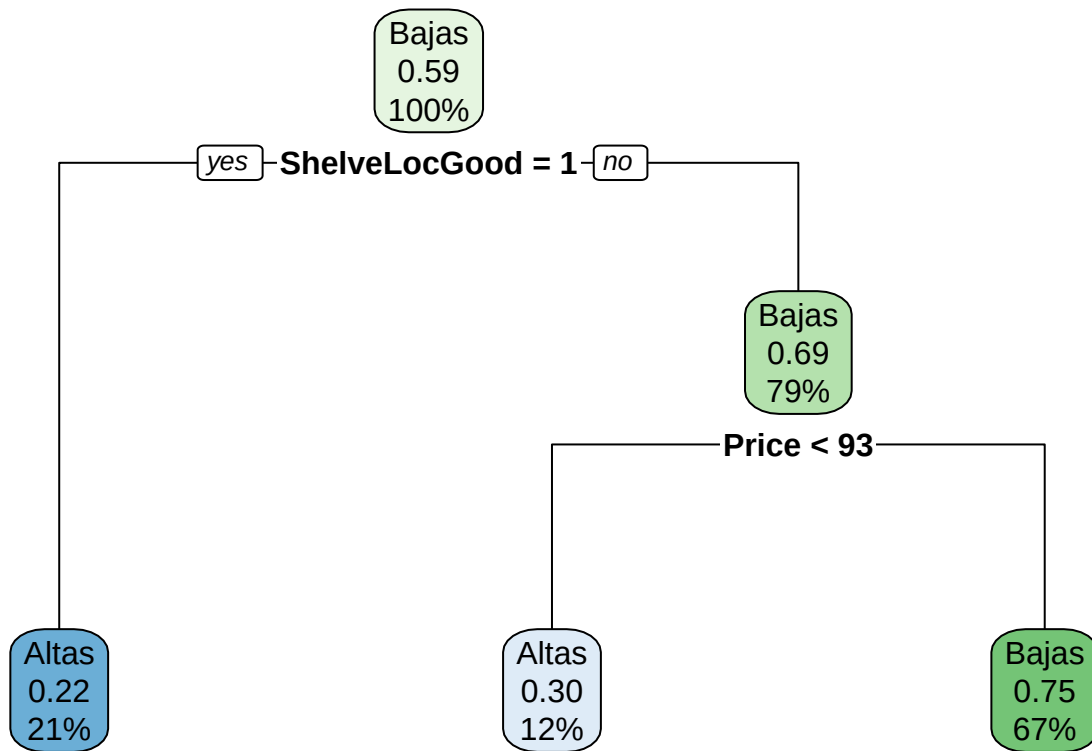
```
ggplot(df_plot, aes(x = variable, y = value, fill = variable)) +  
  geom_boxplot() +  
  labs(title = "Distribución de métricas (K-Fold CV)", x = "Metricas", y = "Valor") +  
  theme_minimal()
```



Tenemos un modelo muy conservador, que puede resultar útil para evitar invertir en tiendas que venderán poco (gracias a su alta especificidad).

Una vez con el modelo realizamos el arbol de decision.

```
rpart.plot(modelo$finalModel)
```



La estrategia lógica que el algoritmo ha ‘aprendido’ para distinguir las tiendas con ventas altas (Altas) de las que tienen ventas bajas (Bajas) es la siguiente:

- En el conjunto de datos total (100 %), el 59 % de los casos presenta ventas bajas.
- El árbol ha determinado que la ubicación del producto en la estantería (ShelveLocGood) es el factor determinante para el éxito de las ventas.
- Si la ubicación es buena, hay un 78 % (1 - 0.22) de probabilidades de que quienes tienen ventas ‘Bajas’ pasen a tener ventas ‘Altas’; el 21 % de las tiendas analizadas (unas 84) se encuentra en este nodo.
- El 79 % restante tiene una ubicación no buena; aquí hay una distinción en función del precio:
- **Precio < 93**: Si el precio es competitivo, el modelo sigue pronosticando ventas «Altas», aunque la ubicación en las estanterías no sea óptima.

Este grupo representa el 12 % de los datos, un 70 % que resultará en ventas ‘Altas’.

- **Precio > 93**: representa la mayoría de los casos (67 %); el 75 % de las ventas serán bajas si tenemos precios altos.

El árbol nos indica que, para obtener ventas altas, hay que cumplir una de estas dos condiciones:

- Tener una ubicación excelente en el estante (ShelveLocGood).
- Si la ubicación no es óptima, mantener un precio muy agresivo (Price < 93).

No se han tenido en cuenta otras variables porque, con un coeficiente de rendimiento $cp = 0.0457$, se ha determinado que solo la ubicación en las estanterías y el precio ofrecen un poder predictivo real, descartando el resto para evitar el riesgo de overfitting.

Por lo tanto, añadir variables no habría mejorado la precisión del modelo de forma significativa sin comprometer su simplicidad.

1.3 Modelo de entrenamiento pt2

Intentemos mejorar la capacidad predictiva del modelo aumentando su complejidad.

```
valoresCP <- expand.grid(cp = seq(0.01,0.05,0.01))
```

```
set.seed(12345)
```

```
modelo2 <- train(Sales ~ .,  
data=Carseats,  
method="rpart",  
metric="ROC",  
trControl=entrenamiento,  
tuneGrid=valoresCP)
```

```
## + Fold01: cp=0.01  
## - Fold01: cp=0.01  
## + Fold02: cp=0.01  
## - Fold02: cp=0.01  
## + Fold03: cp=0.01  
## - Fold03: cp=0.01  
## + Fold04: cp=0.01  
## - Fold04: cp=0.01  
## + Fold05: cp=0.01  
## - Fold05: cp=0.01  
## + Fold06: cp=0.01  
## - Fold06: cp=0.01  
## + Fold07: cp=0.01  
## - Fold07: cp=0.01  
## + Fold08: cp=0.01  
## - Fold08: cp=0.01  
## + Fold09: cp=0.01  
## - Fold09: cp=0.01  
## + Fold10: cp=0.01  
## - Fold10: cp=0.01  
## Aggregating results  
## Selecting tuning parameters  
## Fitting cp = 0.01 on full training set
```

```
print(modelo2)
```

```
## CART  
##  
## 400 samples  
## 10 predictor  
## 2 classes: 'Altas', 'Bajas'  
##  
## No pre-processing  
## Resampling: Cross-Validated (10 fold)  
## Summary of sample sizes: 359, 360, 360, 361, 359, 360, ...  
## Resampling results across tuning parameters:  
##  
##   cp    ROC      Sens      Spec
```

```
## 0.01 0.8054065 0.7077206 0.8222826
## 0.02 0.8043881 0.7194853 0.8137681
## 0.03 0.7549113 0.6463235 0.8266304
## 0.04 0.7404811 0.6529412 0.7932971
## 0.05 0.7231764 0.6055147 0.8137681
##
```

```
## ROC was used to select the optimal model using the largest value.
## The final value used for the model was cp = 0.01.
```

- **ROC (0.805):** La capacidad del modelo para distinguir entre ventas altas y bajas ha mejorado significativamente, pasando de aproximadamente 0.72 a 0.80.
- **Sensibilidad (0.707):** Ahora el modelo es mucho más eficaz a la hora de identificar las tiendas con ventas altas.
- **Especificidad (0.822):** La especificidad sigue siendo muy sólida.

```
# Preparación de los datos: añadimos una columna para distinguir los modelos
df1 <- melt(modelo$resample[, c("ROC", "Sens", "Spec")])
df1$Modelo <- "Modelo 1 (CP 0.045)"

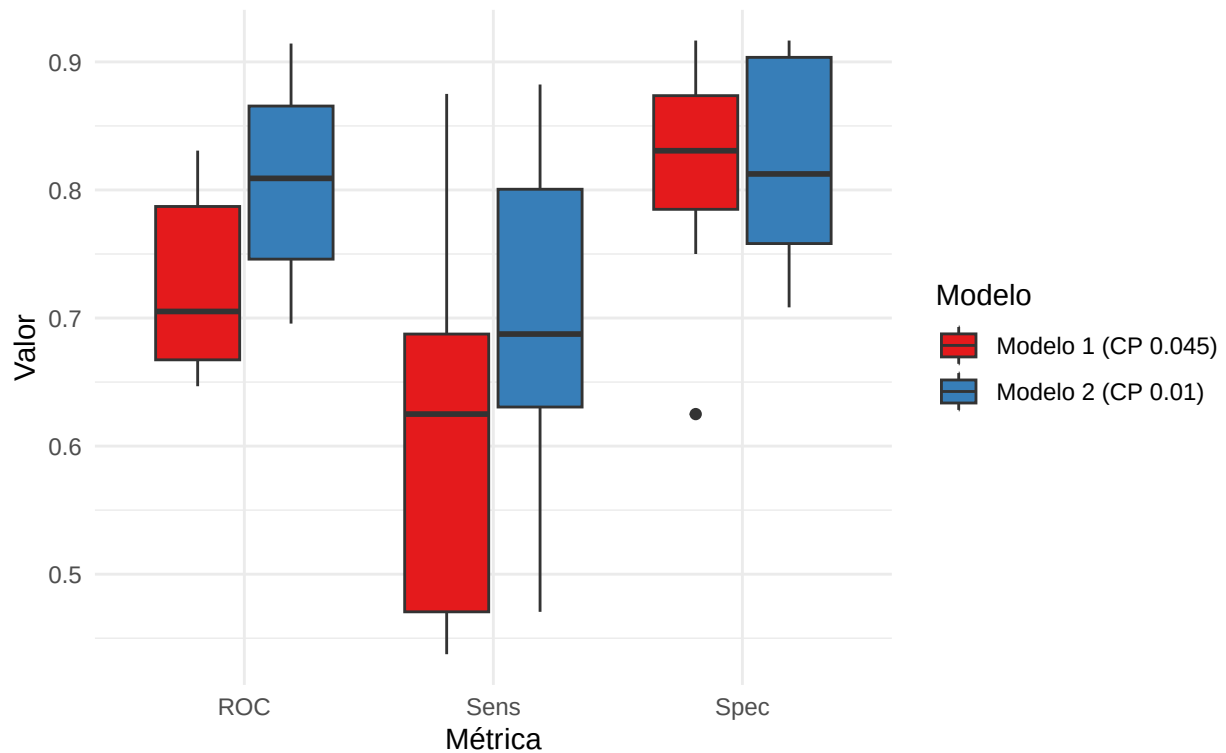
df2 <- melt(modelo2$resample[, c("ROC", "Sens", "Spec")])
df2$Modelo <- "Modelo 2 (CP 0.01)"

# Unión de los dos datasets
df_confronto <- rbind(df1, df2)

# Gráfico de comparación (Box-plots enfrentados)
ggplot(df_confronto, aes(x = variable, y = value, fill = Modelo)) +
  geom_boxplot() +
  labs(title = "Comparación de Rendimiento: Modelo Original vs. Optimizado",
       subtitle = "Mejora de las métricas mediante el Tuning del CP",
       x = "Métrica",
       y = "Valor") +
  scale_fill_manual(values = c("#E41A1C", "#377EB8")) + # Rojo para el 1, Azul para el 2
  theme_minimal()
```

Comparación de Rendimiento: Modelo Original vs. Optimizado

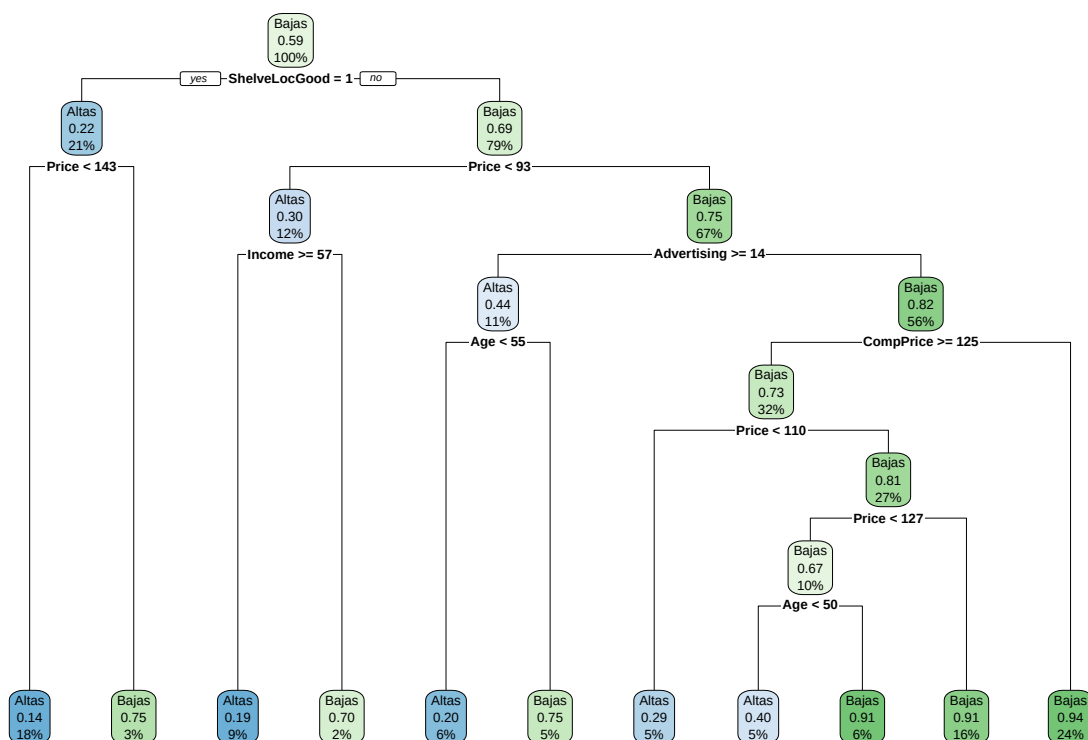
Mejora de las métricas mediante el Tuning del CP



Al aumentar la complejidad, hemos obtenido un modelo que no solo es capaz de identificar quién venderá poco, sino que también se ha vuelto mucho más eficaz a la hora de detectar las tiendas que tendrán éxito.

Albero de decision.

```
#Gráfico del árbol  
rpart.plot(modelo2$finalModel)
```

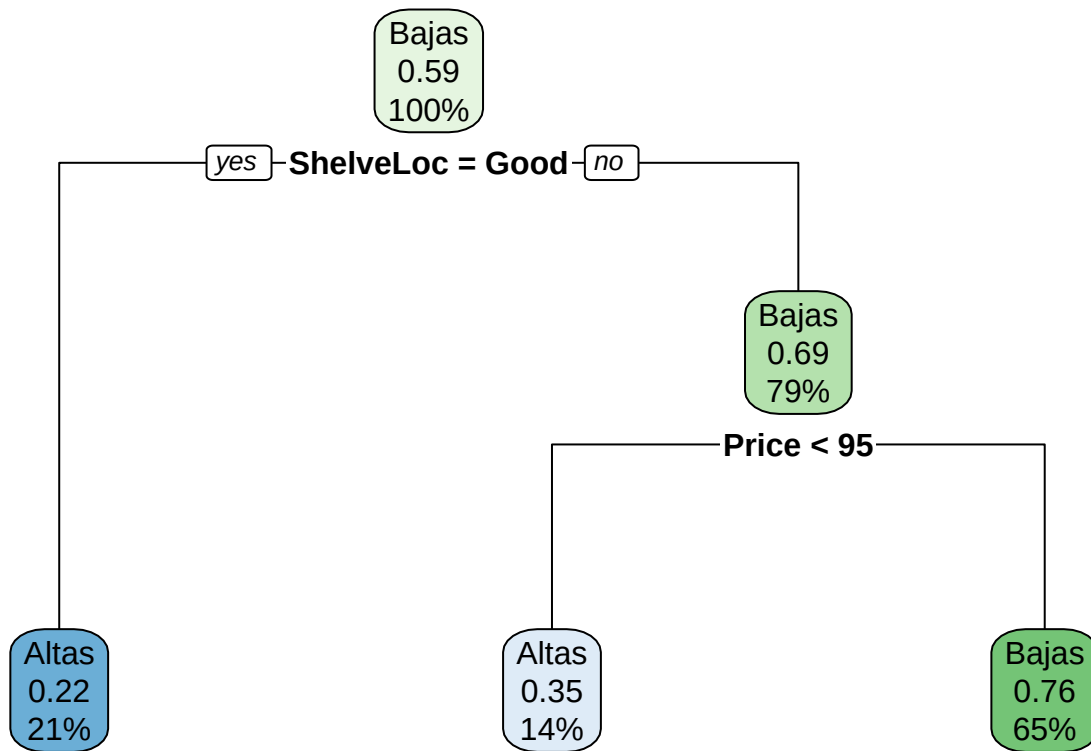



Como podemos observar, se ha vuelto mucho más complejo que antes, al añadir nuevas variables que antes no se consideraban significativas.

En este momento, el árbol tiene muchas ramas que representan muy pocas tiendas, lo que lo hace más propenso al **overfitting**.

Hagamos que el árbol sea mucho más corto y robusto, imponiendo un número mínimo de observaciones (tiendas) que deben estar presentes en cada nodo final para que la rama sea válida.

```
#Reducimos el tamaño para facilitar la interpretación
arbol <- rpart(Sales ~ ., data=Carseats,
cp= 0.01, control = rpart.control(minbucket = 50))
rpart.plot(arbol)
```



En general, no ha cambiado mucho con respecto al árbol anterior.

En ambos modelos, la ubicación en la estantería ($\text{ShelveLoc} = \text{Good}$) es la variable más importante para predecir unas ventas elevadas.

La primera parte se mantiene sin cambios; lo que varía ligeramente es el umbral de precio (de 93 a 95).

Las tiendas que tienen un precio competitivo han pasado del 12 % al 14 % del total; también desciende la probabilidad de que las ventas sean ‘Altas’ (del 70 % al 65 %).

La probabilidad de ventas ‘Bajas’ se ha mantenido casi idéntica (del 75 % al 76 %).

```

# Reglas detalladas con cobertura y estilo en columnas
print(rpart.rules(arbol,
                  style = "tall",      # Hace que las reglas sean más legibles verticalmente
                  cover = TRUE,        # Muestra el porcentaje de datos cubiertos por cada regla
                  nn = TRUE))          # Muestra el número del nodo correspondiente

## [2] Sales is 0.22 with cover 21% when
##      ShelveLoc is Good
##
## [6] Sales is 0.35 with cover 14% when
##      ShelveLoc is Bad or Medium
##      Price < 95
##
## [7] Sales is 0.76 with cover 65% when
##      ShelveLoc is Bad or Medium
##      Price >= 95

```

1.4 Cálculo de las predicciones sobre el conjunto de datos

```
predichos <- predict(arbol, type='class')
confusionMatrix(predichos, Carseats$Sales)

## Confusion Matrix and Statistics
##
##              Reference
## Prediction Altas Bajas
##      Altas   101    38
##      Bajas    63   198
##
##              Accuracy : 0.7475
##              95% CI : (0.7019, 0.7894)
##      No Information Rate : 0.59
##      P-Value [Acc > NIR] : 3.036e-11
##
##              Kappa : 0.4657
##
##  Mcnemar's Test P-Value : 0.01694
##
##              Sensitivity : 0.6159
##              Specificity : 0.8390
##              Pos Pred Value : 0.7266
##              Neg Pred Value : 0.7586
##              Prevalence : 0.4100
##              Detection Rate : 0.2525
##      Detection Prevalence : 0.3475
##              Balanced Accuracy : 0.7274
##
##              'Positive' Class : Altas
##
```

El modelo ha acertado la clase correcta en aproximadamente el 75 % de los casos.

El P-Value [Acc > NIR] nos confirma estadísticamente que el modelo es significativo y más útil que el caso.

Al trabajar con un modelo más robusto, observamos que la sensibilidad ha disminuido: el modelo solo logra identificar el 62 % de las tiendas con ventas elevadas.

En cambio, al analizar la especificidad, podemos observar que identifica correctamente el 84 % de las tiendas con ventas bajas.

El árbol es un excelente **filtro prudente**, muy bueno para identificar qué tiendas evitarán la quiebra (alta especificidad), pero aún es un poco tímido a la hora de detectar todas las oportunidades de éxito (sensibilidad mejorable).

Al analizar el valor p de el test de McNemar, que resulta inferior a 0.05, se observa que los errores del modelo no son aleatorios.

Observamos que los falsos negativos (63) son casi el doble que los falsos positivos (38).

Podemos concluir confirmando que el modelo es “**pesimista**”.

Tiende a subestimar el potencial de las tiendas, prefiriendo clasificar una tienda de éxito como “Bajas” antes que arriesgarse a calificar de “Altas” a una tienda mediocre.

Ejercicio 2

El set de datos **Boston** disponible en el paquete MASS contiene precios de viviendas de la ciudad de Boston, así como información socioeconómica del barrio en el que se encuentran. Se pretende ajustar un modelo de regresión que permita predecir el precio medio de una vivienda (**medv**) en función de las variables disponibles.

2.1. Carga y exploración inicial de los datos

El objetivo principal de este análisis es predecir el **valor mediano de las viviendas (medv)** en función de diversas variables socioeconómicas y del entorno.

Variables principales:

- **medv**: (Variable objetivo) Valor mediano de las casas ocupadas por sus dueños (en miles de dólares).
- **rm**: Número promedio de habitaciones por vivienda.
- **lstat**: Porcentaje de la población con un estatus socioeconómico bajo.
- **crim**: Tasa de criminalidad per cápita por ciudad.
- **age**: Proporción de unidades ocupadas por sus dueños construidas antes de 1940.
- **dis**: Distancias ponderadas a cinco centros de empleo de Boston.
- **nox**: Concentración de óxidos de nitrógeno (indicador de contaminación).

En este ejercicio, utilizaremos un enfoque de **árboles de decisión** para capturar las relaciones no lineales entre estas variables y el precio de las viviendas, optimizando el modelo mediante **validación cruzada** y técnicas de **poda (pruning)** para garantizar que los resultados sean generalizables a nuevos datos.

```
# Mostrar las primeras 6 filas del conjunto de datos para una inspección rápida
head(Boston)
```

```
##      crim zn indus chas   nox    rm age   dis rad tax ptratio  black lstat
## 1 0.00632 18  2.31    0 0.538 6.575 65.2 4.0900   1 296    15.3 396.90  4.98
## 2 0.02731  0  7.07    0 0.469 6.421 78.9 4.9671   2 242    17.8 396.90  9.14
## 3 0.02729  0  7.07    0 0.469 7.185 61.1 4.9671   2 242    17.8 392.83  4.03
## 4 0.03237  0  2.18    0 0.458 6.998 45.8 6.0622   3 222    18.7 394.63  2.94
## 5 0.06905  0  2.18    0 0.458 7.147 54.2 6.0622   3 222    18.7 396.90  5.33
## 6 0.02985  0  2.18    0 0.458 6.430 58.7 6.0622   3 222    18.7 394.12  5.21
##   medv
## 1 24.0
## 2 21.6
## 3 34.7
## 4 33.4
## 5 36.2
## 6 28.7
```

```
# Verificar si existen valores ausentes (NAs) en las columnas
# colSums suma los valores lógicos; pander le da un formato de tabla limpia al reporte
pander(colSums(is.na(Boston)))
```

Table 5: Table continues below

crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	black
0	0	0	0	0	0	0	0	0	0	0	0

lstat	medv
0	0

```
# Visualizar la estructura del dataframe (tipo de variables, número de observaciones y dimensiones)
str(Boston)
```

```
## 'data.frame':    506 obs. of  14 variables:
## $ crim      : num  0.00632 0.02731 0.02729 0.03237 0.06905 ...
## $ zn        : num  18 0 0 0 0 12.5 12.5 12.5 12.5 ...
## $ indus     : num  2.31 7.07 7.07 2.18 2.18 2.18 7.87 7.87 7.87 7.87 ...
## $ chas      : int   0 0 0 0 0 0 0 0 0 0 ...
## $ nox       : num  0.538 0.469 0.469 0.458 0.458 0.458 0.524 0.524 0.524 0.524 ...
## $ rm        : num  6.58 6.42 7.18 7 7.15 ...
## $ age       : num  65.2 78.9 61.1 45.8 54.2 58.7 66.6 96.1 100 85.9 ...
## $ dis       : num  4.09 4.97 4.97 6.06 6.06 ...
## $ rad       : int   1 2 2 3 3 3 5 5 5 ...
## $ tax       : num  296 242 242 222 222 222 311 311 311 311 ...
## $ ptratio   : num  15.3 17.8 17.8 18.7 18.7 18.7 15.2 15.2 15.2 15.2 ...
## $ black     : num  397 397 393 395 397 ...
## $ lstat     : num  4.98 9.14 4.03 2.94 5.33 ...
## $ medv      : num  24 21.6 34.7 33.4 36.2 28.7 22.9 27.1 16.5 18.9 ...
```

```
# Generar un resumen estadístico descriptivo (mínimo, máximo, media, cuartiles)
# de todas las variables del dataset
pander(summary(Boston))
```

Table 7: Table continues below

crim	zn	indus	chas
Min. : 0.00632	Min. : 0.00	Min. : 0.46	Min. :0.00000
1st Qu.: 0.08205	1st Qu.: 0.00	1st Qu.: 5.19	1st Qu.:0.00000
Median : 0.25651	Median : 0.00	Median : 9.69	Median :0.00000
Mean : 3.61352	Mean : 11.36	Mean :11.14	Mean :0.06917
3rd Qu.: 3.67708	3rd Qu.: 12.50	3rd Qu.:18.10	3rd Qu.:0.00000
Max. :88.97620	Max. :100.00	Max. :27.74	Max. :1.00000

Table 8: Table continues below

nox	rm	age	dis
Min. :0.3850	Min. :3.561	Min. : 2.90	Min. : 1.130
1st Qu.:0.4490	1st Qu.:5.886	1st Qu.: 45.02	1st Qu.: 2.100
Median :0.5380	Median :6.208	Median : 77.50	Median : 3.207
Mean :0.5547	Mean :6.285	Mean : 68.57	Mean : 3.795
3rd Qu.:0.6240	3rd Qu.:6.623	3rd Qu.: 94.08	3rd Qu.: 5.188
Max. :0.8710	Max. :8.780	Max. :100.00	Max. :12.127

Table 9: Table continues below

rad	tax	ptratio	black
Min. : 1.000	Min. :187.0	Min. :12.60	Min. : 0.32
1st Qu.: 4.000	1st Qu.:279.0	1st Qu.:17.40	1st Qu.:375.38
Median : 5.000	Median :330.0	Median :19.05	Median :391.44
Mean : 9.549	Mean :408.2	Mean :18.46	Mean :356.67
3rd Qu.:24.000	3rd Qu.:666.0	3rd Qu.:20.20	3rd Qu.:396.23

rad	tax	ptratio	black
Max. :24.000	Max. :711.0	Max. :22.00	Max. :396.90

lstat	medv
Min. : 1.73	Min. : 5.00
1st Qu.: 6.95	1st Qu.:17.02
Median :11.36	Median :21.20
Mean :12.65	Mean :22.53
3rd Qu.:16.95	3rd Qu.:25.00
Max. :37.97	Max. :50.00

2.2. Análisis Exploratorio de Datos (EDA)

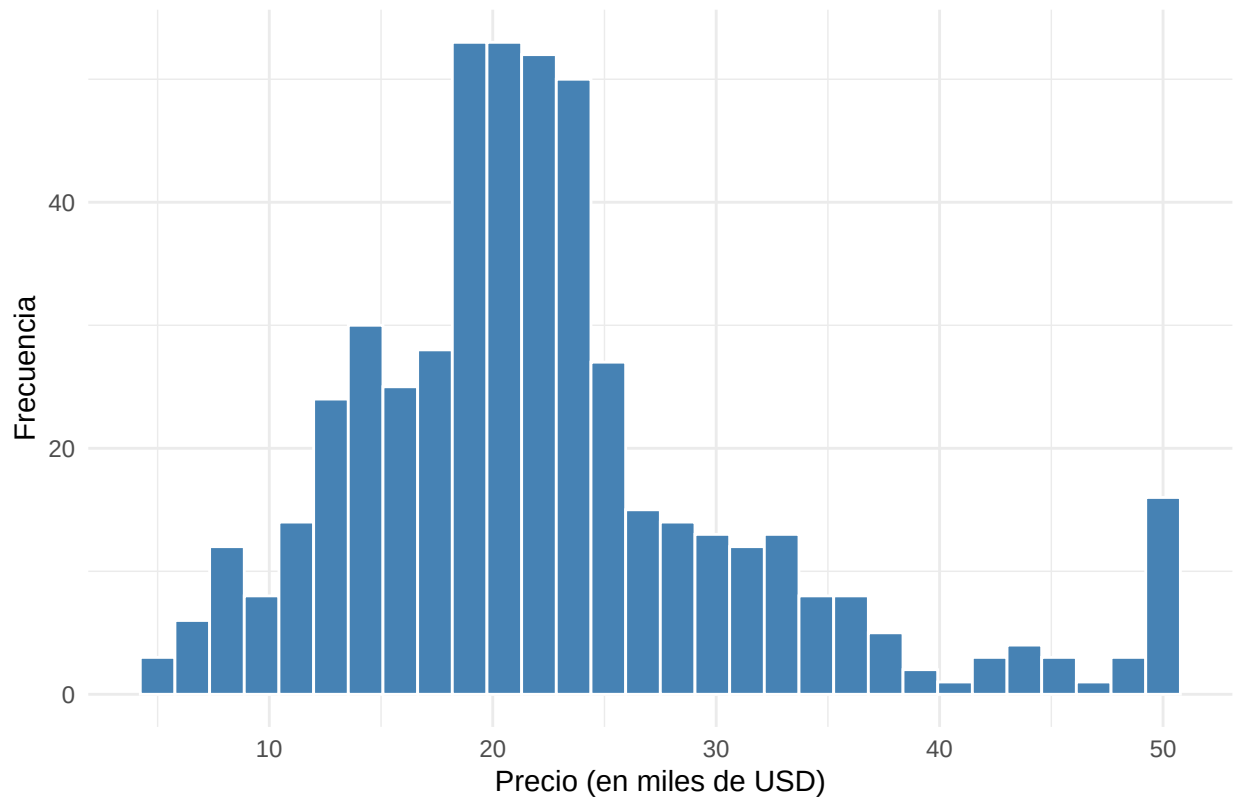
Antes de entrenar el modelo, es fundamental realizar un análisis exploratorio para entender las relaciones entre las variables y cómo afectan al precio de la vivienda (**medv**). A diferencia de los modelos lineales, los árboles de decisión pueden capturar relaciones no lineales y complejas, por lo que buscaremos patrones visuales y correlaciones fuertes.

Visualización de la Variable Objetivo

Primero, analizamos la distribución de los precios para ver si existen sesgos o valores atípicos.

```
# Histograma del precio mediano (medv)
ggplot(Boston, aes(x = medv)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "white") +
  labs(title = "Distribución del Precio Mediano de la Vivienda",
        x = "Precio (en miles de USD)",
        y = "Frecuencia") +
  theme_minimal()
```

Distribución del Precio Mediano de la Vivienda



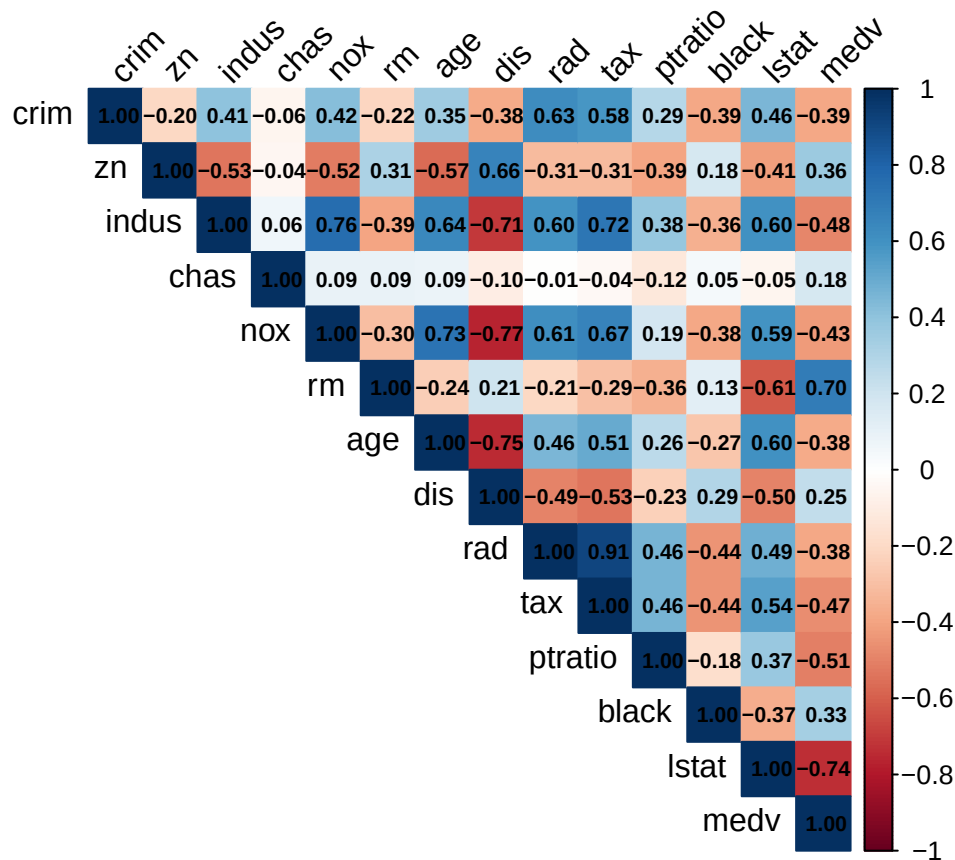
El histograma muestra que la mayoría de las viviendas se encuentran en el rango de 15,000 a 25,000 USD. Se observa un pico inusual en los 50,000 USD, lo que sugiere que los valores podrían haber sido truncados en ese límite superior.

Análisis de Correlación

Identificamos qué variables tienen una relación más estrecha con el precio. Esto nos da una pista de qué variables aparecerán en los niveles superiores de nuestro árbol de decisión.

```
# Calcular la matriz de correlación
cor_matrix <- cor(Boston)

# Visualizar la matriz
corrplot(cor_matrix, method = "color", type = "upper",
          tl.col = "black", tl.srt = 45,
          addCoef.col = "black", number.cex = 0.7)
```



La matriz de correlación indica que el precio (medv) tiene una fuerte correlación positiva con el número de habitaciones (rm) y una fuerte correlación negativa con el estatus socioeconómico bajo (lstat). Estas dos variables son, probablemente, los predictores más importantes para nuestro modelo.

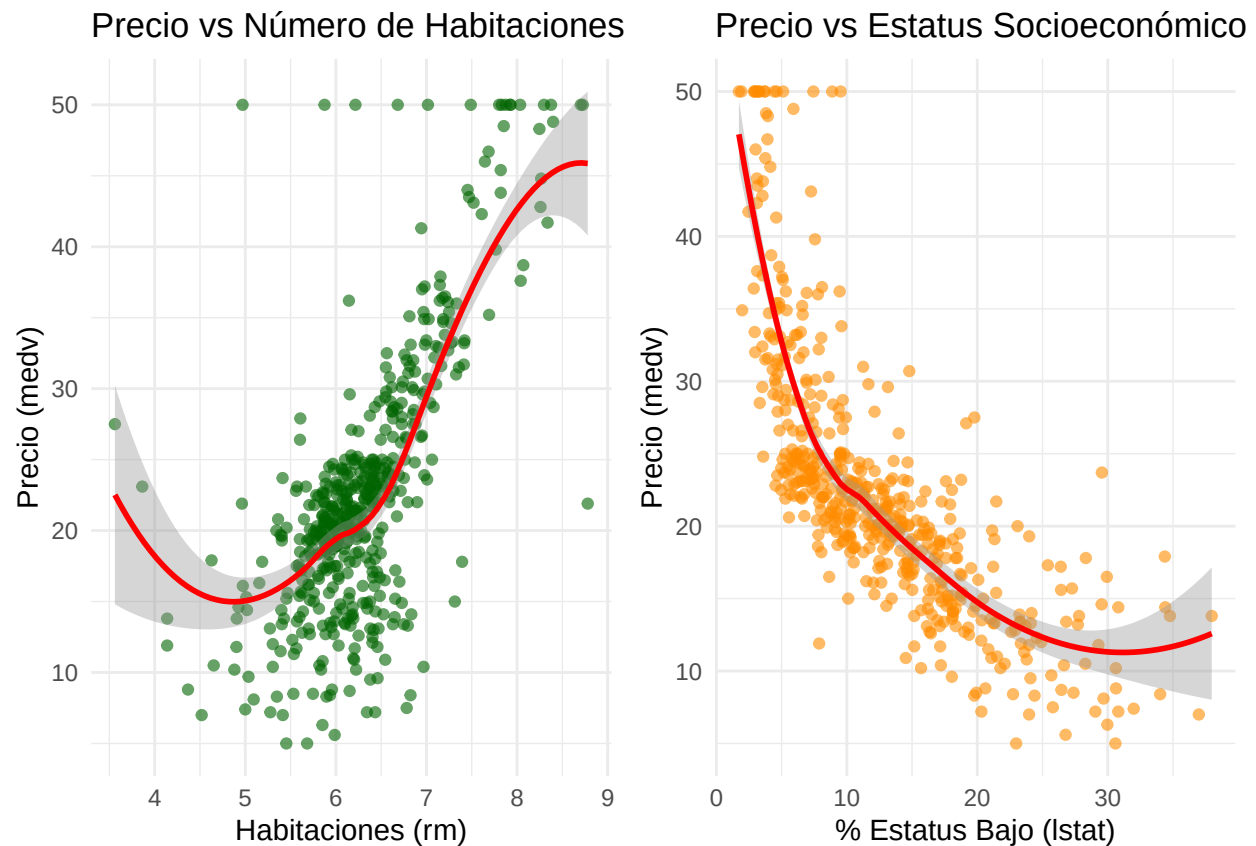
Relación entre el Precio y Variables Clave

Visualizamos las dos variables con mayor impacto mediante diagramas de dispersión.

```
# Relación con el número de habitaciones (rm)
p1 <- ggplot(Boston, aes(x = rm, y = medv)) +
  geom_point(alpha = 0.6, color = "darkgreen") +
  geom_smooth(method = "loess", color = "red") +
  labs(title = "Precio vs Número de Habitaciones",
       x = "Habitaciones (rm)",
       y = "Precio (medv)") +
  theme_minimal()

# Relación con el estatus socioeconómico (lstat)
p2 <- ggplot(Boston, aes(x = lstat, y = medv)) +
  geom_point(alpha = 0.6, color = "darkorange") +
  geom_smooth(method = "loess", color = "red") +
  labs(title = "Precio vs Estatus Socioeconómico",
       x = "% Estatus Bajo (lstat)",
       y = "Precio (medv)") +
  theme_minimal()

# library(gridExtra)
grid.arrange(p1, p2, ncol = 2)
```

- **Habitaciones (rm):** Se observa una relación lineal clara; a más habitaciones, mayor es el precio.
- **Estatus (lstat):** La relación es inversamente proporcional y parece ser no lineal (curva), lo cual justifica el uso de un árbol de decisión, ya que estos modelos manejan bien este tipo de curvaturas sin necesidad de transformaciones matemáticas complejas.

2.3.1. División del conjunto de datos

Para garantizar que nuestro modelo sea capaz de generalizar sus predicciones a nuevos datos, dividimos el conjunto original en dos grupos: **Entrenamiento (80%)** y **Prueba (20%)**. El modelo aprenderá los patrones de los datos de entrenamiento y, posteriormente, evaluaremos su precisión con el grupo de prueba, el cual funciona como datos “no vistos”.

Utilizamos la función `createDataPartition` del paquete `caret`, ya que esta realiza un muestreo estratificado, manteniendo la distribución de la variable objetivo (`medv`) en ambos subconjuntos.

```
set.seed(123)

# Crear el índice de partición (80% para entrenamiento)
train_index <- createDataPartition(Boston$medv, p = 0.8, list = FALSE)

# Dividir los datos
train_data <- Boston[train_index, ]
test_data  <- Boston[-train_index, ]

# Verificar dimensiones
```

```
cat("Registros en entrenamiento:", nrow(train_data), "\n")
```

```
## Registros en entrenamiento: 407
```

```
cat("Registros en prueba:", nrow(test_data), "\n")
```

```
## Registros en prueba: 99
```

2.3.2. Configuración de la Validación Cruzada

Dentro del conjunto de entrenamiento, no basta con ajustar un solo árbol, ya que podríamos caer en el **sobreaajuste (overfitting)**. Para encontrar el nivel de complejidad óptimo del árbol (la “poda” ideal), utilizaremos **Validación Cruzada de 10 pliegues (10-fold Cross-Validation)**.

Este proceso divide el grupo de entrenamiento en 10 partes, entrena el modelo en 9 y lo valida en la restante, repitiendo el proceso 10 veces. Esto nos permite encontrar el **Parámetro de Complejidad (CP)** que minimiza el error de predicción.

```
set.seed(2137)
```

```
# Definir el esquema de entrenamiento: Validación Cruzada de 10 pliegues
train_control <- trainControl(
  method = "cv",          # Cross-validation
  number = 10             # Número de pliegues (folds)
)
```

2.4. Construcción del modelo

En esta etapa entrenamos el árbol de regresión utilizando el paquete **caret**. El modelo busca automáticamente el **Parámetro de Complejidad (CP)** óptimo evaluando 20 valores diferentes (**tuneLength = 20**) para encontrar el nivel de poda que minimice el error (RMSE).

```
# Entrenamiento del modelo con optimización de hiperparámetros
cv_tree <- train(
  medv ~ .,
  data = train_data,
  method = "rpart",
  trControl = train_control,
  tuneLength = 20,
  metric = "RMSE"
)

# Mostrar el mejor valor de CP seleccionado
print(cv_tree$bestTune)
```

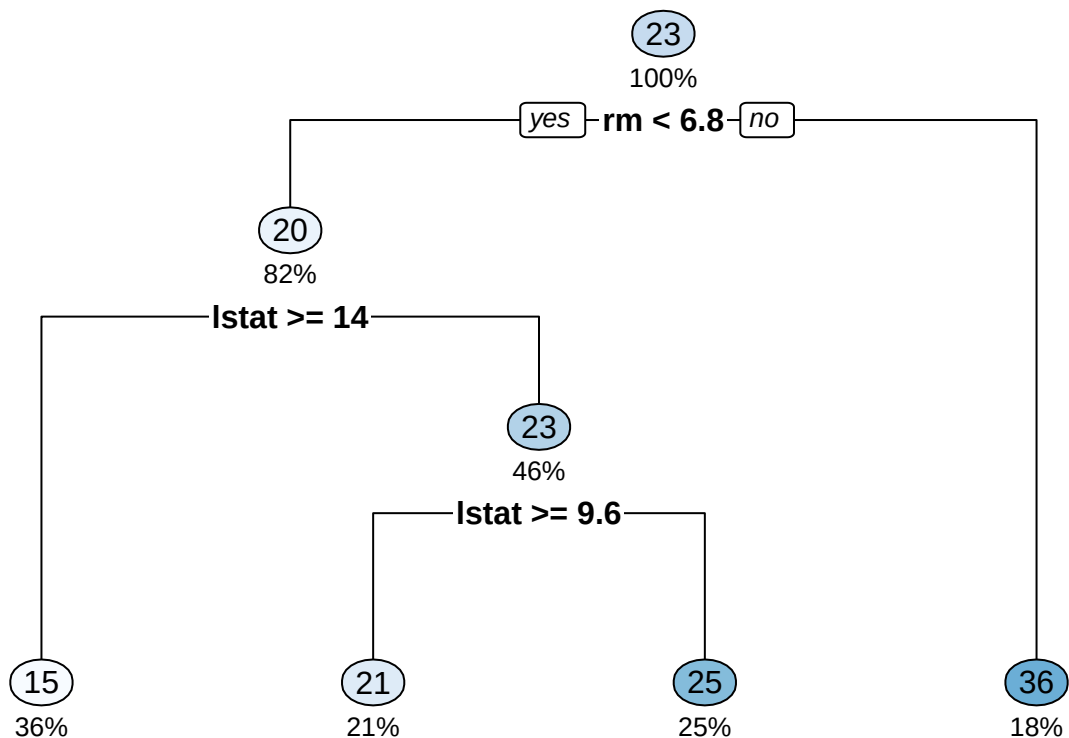
```
##               cp
## 3 0.002166586
```

El resultado muestra el valor de CP que optimiza el modelo. Un CP menor permite árboles más complejos, mientras que uno mayor penaliza el crecimiento para evitar el sobreajuste.

2.5. Visualización del árbol

Representamos gráficamente el árbol final optimizado. Cada nodo muestra la variable que genera la división y el precio promedio predicho (**medv**) para las viviendas en ese segmento.

```
# Visualización del modelo final podado
library(rpart.plot)
```

2.6. Evaluación del modelo (Predicción)

Finalmente, evaluamos el modelo utilizando el conjunto de prueba (datos no vistos). Calculamos el **RMSE** para medir el error promedio de las predicciones y el **R-cuadrado** para determinar qué porcentaje de la variabilidad del precio logra explicar nuestro árbol.

```
# Predicción con los datos de prueba
final_predictions <- predict(cv_tree, newdata = test_data)
final_predictions2 <- predict(arbol2, newdata = test_data)
```

```
# Cálculo de métricas de rendimiento
test_rmse <- RMSE(final_predictions, test_data$medv)
test_r2 <- R2(final_predictions, test_data$medv)
```

```
test_rmse2 <- RMSE(final_predictions2, test_data$medv)
test_r2_2 <- R2(final_predictions2, test_data$medv)
```

```
# Mostrar resultados
cat("RMSE final en prueba:", test_rmse, "\n")
```

```
## RMSE final en prueba: 4.258817
```

```
cat("R-cuadrado final en prueba:", test_r2, "\n")
```

```
## R-cuadrado final en prueba: 0.7924287
```

```
# Mostrar resultados2
cat("RMSE final en arbol mas facil:", test_rmse2, "\n")
```

```
## RMSE final en arbol mas facil: 5.416295
```

```
cat("R-cuadrado final en arbol mas facil:", test_r2_2, "\n")
```

```
## R-cuadrado final en arbol mas facil: 0.6756758
```

El **RMSE** nos indica cuánto se desvía, en promedio, nuestra predicción del precio real (en miles de USD).
El **R-cuadrado** muestra la precisión global; cuanto más cercano a 1, mejor es el ajuste del modelo.